Alia 4 LED Controller - Tutorial

This guide gets you from zero to a running board, then walks through customizing LED patterns — starting with the easiest possible option.

Part 1: Setup

No Arduino experience needed — just follow these in order. This works fine on a Windows, Mac, or Linux computer. It's difficult to do on a Chromebook, though — if that's all you have, try sharing a laptop with another student for this project.

1. **Install Arduino IDE 2.x** — the free program you'll use to load code onto the board. Download it from arduino.cc and install it like any other app.
2. **Install the board core** — this teaches Arduino IDE how to talk to your specific chip. This tutorial covers the **RP2040** — the chip used in this example (either the Seeed XIAO RP2040 or Adafruit QT Py RP2040 board). Other chips are supported too; see README.md if that's what you have.
    - Tools → Board → Boards Manager
    - Search `rp2040`, install "Raspberry Pi Pico/RP2040" (v5.4.3+)
3. **Install the library** — Tools → Manage Libraries → search `Adafruit_NeoPixel` → Install (v1.15.2+). This is the ready-made code the sketch relies on to control the LEDs.
4. **Get the code** — this is the one file that makes the LEDs do anything:
    - Open alia_blinky.ino → right-click → **Save As** → save it as `alia_blinky.ino`
    - That's all you need to get a board blinking. Want the PCB design files, soldering photos, or full source history too? Visit github.com/johncohn/alia_blinky → green **Code** button → **Download ZIP** (or `git clone`) instead.
5. **Open and upload**
    - File → Open → the `alia_blinky.ino` you saved in step 4
    - Tools → Board → Raspberry Pi RP2040 Boards → select your exact board (Seeed XIAO RP2040 or Adafruit QT Py RP2040)
    - Plug the board into your computer with a USB cable, then Tools → Port → select it (usually the only new entry in the list)
    - Click **Upload** (→ button, top left) — this compiles the code and sends it to the board
    - You should now see the lights blinking, sequencing through all the patterns. From now on, whenever you plug the board into power, it will

run these patterns — until you change the code and upload again.

6. **Watch the output of the code** — Tools → Serial Monitor opens a window that shows text the board sends back over USB. Set its baud rate (bottom-right dropdown) to **115200**. You should see the board type it detected and the name of each pattern as it cycles through them.

Troubleshooting: if upload fails, double-tap the board's RESET button to force bootloader mode (a drive named `RPI-RP2` should appear), then upload again.

Part 2: Customize the CUSTOM Pattern (the easy way)

The board automatically cycles through several light patterns (this is what "auto-cycle" means throughout this guide). One of them is called **CUSTOM**. By default it's a simple pixel-by-pixel color wipe — it lights the string one LED at a time in red, then does the same in green, then blue, then loops.

We wrote CUSTOM so it's easy to change: either edit its code directly, or copy in one of the ready-made samples from the Sample patterns section below. (Part 3 covers how to add even more patterns alongside it.)

Try a sample first

1. Open `alia_blinky.ino` in Arduino IDE and find the function `customPattern()` (search for it with Ctrl+F / Cmd+F).
2. Pick a sample from the Sample patterns section below and copy everything **inside its `{ }`** over the code inside `customPattern()`'s `{ }`. Leave the function's own name (`void customPattern()`) alone — only its body changes.
3. Upload, open Serial Monitor (115200 baud), and wait for `CUSTOM` to print — that's your new pattern running in that slot.

New to this? **Dot Chase** or **Sparkle** are the easiest to follow — start with one of those. Once copying a sample in feels easy, try tweaking values inside it (colors, timing numbers) before moving on to writing something from scratch in Part 3.

Sample patterns

Roughly easiest to most advanced — pick any of them, in any order.

Dot Chase — one white LED travels down the entire string (all 4 props, then the tail) and loops:

```
  static int currentLED = 0;
  static unsigned long lastUpdate = 0;

  if (millis() - lastUpdate > 30) {
    strip.clear();
    strip.setPixelColor(currentLED, strip.Color(brightness, brig
    strip.show();
    lights();

    currentLED++;
    if (currentLED >= LED_COUNT) currentLED = 0;
    lastUpdate = millis();
  }
```

Sparkle — random LEDs twinkle white against a dark background:

```
  static unsigned long lastUpdate = 0;

  if (millis() - lastUpdate > 50) {
    strip.clear();
    for (int i = 0; i < 6; i++) {  // 6 sparkles at a time
      strip.setPixelColor(random(LED_COUNT), strip.Color(brightr
    }
    strip.show();
    lights();
    lastUpdate = millis();
  }
```

Red, White & Blue — marching stripes down the whole string:

```
  static int offset = 0;
  static unsigned long lastUpdate = 0;

  if (millis() - lastUpdate > 100) {
    for (int i = 0; i < LED_COUNT; i++) {
      int stripe = (i + offset) % 3;  // cycles 0, 1, 2, 0, 1, 2
      if (stripe == 0) strip.setPixelColor(i, strip.Color(bright
      else if (stripe == 1) strip.setPixelColor(i, strip.Color(b
      else strip.setPixelColor(i, strip.Color(0, 0, brightness))
    }
    strip.show();
    lights();
    offset++;
    lastUpdate = millis();
  }
```

Breathing — the whole string smoothly fades up and down in one color:

```
  static int level = 0;
  static int direction = 5;
  static unsigned long lastUpdate = 0;

  if (millis() - lastUpdate > 20) {
    level += direction;
    if (level >= 255) { level = 255; direction = -5; }
    else if (level <= 0) { level = 0; direction = 5; }

    for (int i = 0; i < LED_COUNT; i++) {
      strip.setPixelColor(i, strip.Color(0, 0, level));  // Blue
    }
    strip.show();
    lights();
    lastUpdate = millis();
  }
```

Fire — random orange/red flicker across all LEDs:

```
  static unsigned long lastUpdate = 0;

  if (millis() - lastUpdate > 40) {
    for (int i = 0; i < LED_COUNT; i++) {
      int flicker = random(brightness / 2, brightness);
      strip.setPixelColor(i, strip.Color(flicker, flicker / 4, 0
    }
    strip.show();
    lights();
    lastUpdate = millis();
  }
```

Comet — a bright head with a fading trail travels down the string:

```
  static int headPos = 0;
  static unsigned long lastUpdate = 0;
  const int trailLength = 6;

  if (millis() - lastUpdate > 30) {
    strip.clear();
    for (int t = 0; t < trailLength; t++) {
      int pos = headPos - t;
      if (pos >= 0 && pos < LED_COUNT) {
        int fade = brightness - (t * (brightness / trailLength));
        if (fade < 0) fade = 0;
        strip.setPixelColor(pos, strip.Color(fade, fade, fade));
      }
    }
    strip.show();
    lights();
    headPos++;
    if (headPos >= LED_COUNT + trailLength) headPos = 0;
    lastUpdate = millis();
  }
```

Alternating Props — props 1 & 3 and props 2 & 4 swap colors every half second:

```
  static bool toggle = false;
  static unsigned long lastUpdate = 0;

  if (millis() - lastUpdate > 500) {
    uint32_t colorA, colorB;
    if (toggle) {
      colorA = strip.Color(brightness, 0, 0);  // Red
      colorB = strip.Color(0, 0, brightness);  // Blue
    } else {
      colorA = strip.Color(0, 0, brightness);  // Blue
      colorB = strip.Color(brightness, 0, 0);  // Red
    }

    for (int led = 0; led < 9; led++) {
      strip.setPixelColor(PROP1_START + led, colorA);
      strip.setPixelColor(PROP3_START + led, colorA);
      strip.setPixelColor(PROP2_START + led, colorB);
      strip.setPixelColor(PROP4_START + led, colorB);
    }
    strip.show();
    lights();
    toggle = !toggle;
    lastUpdate = millis();
  }
```

Rainbow Props (Solid) — each prop is a solid color; all 4 slowly cycle through the rainbow, offset so no two props ever match. `ColorHSV`/`gamma32` are NeoPixel library helpers that turn a hue angle (0-65535, i.e. one full trip around the color wheel) into a properly-corrected RGB color:

```
static uint16_t baseHue = 0;
static unsigned long lastUpdate = 0;

if (millis() - lastUpdate > 20) {
  uint16_t hueStep = 65536 / 4;  // split the color wheel 4 wa

  for (int prop = 0; prop < 4; prop++) {
    uint32_t color = strip.gamma32(strip.ColorHSV(baseHue + (p
    for (int led = 0; led < 9; led++) {
      strip.setPixelColor((prop * 9) + led, color);
    }
  }
  strip.show();
  lights();
  baseHue += 100;
  lastUpdate = millis();
}
```

Reference: what CUSTOM looks like by default

You don't need to understand this to try a sample above — it's here in case you're curious what you're replacing:

```
void customPattern() {
  static int ledIndex = 0;
  static int colorIndex = 0;
  static unsigned long lastUpdate = 0;

  // Basic colors to cycle through - EDIT ME!
  uint32_t colors[] = {
    strip.Color(brightness, 0, 0),  // Red
    strip.Color(0, brightness, 0),  // Green
    strip.Color(0, 0, brightness)   // Blue
  };
  int numColors = 3;

  if (millis() - lastUpdate > 30) {
    strip.setPixelColor(ledIndex, colors[colorIndex]);
    strip.show();
    lights();

    ledIndex++;
    if (ledIndex >= LED_COUNT) {
      ledIndex = 0;
      colorIndex = (colorIndex + 1) % numColors;
      strip.clear();
    }

    lastUpdate = millis();
  }
}
```

Part 3: Writing an Additional Pattern From Scratch

Once you're comfortable editing `customPattern()`, you might want a whole new pattern running *alongside* CUSTOM instead of replacing it. That needs 4 edits instead of 1 — here's what each one does and why it's needed:

#	What	Where	Why
1	Write your pattern function	CUSTOM PATTERNS SECTION (~line 1062)	This is where you tell the chip what your new pattern should actually do
2	Add one to NUM_PATTERNS	~line 78	This tells the chip there's one more pattern in the sequence to cycle through

3	Add a `case` for it in the `switch`	`loop()` (~line 1194)	This tells the chip to actually run your new function when the cycle reaches that pattern number
4	Add its name to `subModeNames[]`	~line 1179	This gives your pattern a readable name that prints to Serial Monitor, so you can confirm it's running

Rules of thumb: `case` numbers must be sequential starting at 0 with no gaps, and `NUM_PATTERNS` must equal the total count. The 5 built-in patterns (including CUSTOM) already occupy cases 0-4, so your first additional pattern is **case 5**.

Worked Example: Red, White & Blue as its own pattern

Step 1 — write the function. Paste into the `CUSTOM PATTERNS SECTION` — this defines what the pattern actually does:

```
void redWhiteBluePattern() {
  static int offset = 0;
  static unsigned long lastUpdate = 0;

  if (millis() - lastUpdate > 100) {
    for (int i = 0; i < LED_COUNT; i++) {
      int stripe = (i + offset) % 3;
      if (stripe == 0) strip.setPixelColor(i, strip.Color(brig
      else if (stripe == 1) strip.setPixelColor(i, strip.Color
      else strip.setPixelColor(i, strip.Color(0, 0, brightness
    }
    strip.show();
    lights();
    offset++;
    lastUpdate = millis();
  }
}
```

Step 2 — bump the pattern count. At ~line 78, change `NUM_PATTERNS` from 5 to 6 — this tells the chip there's now one more pattern in the sequence.

Step 3 — add a case. In the `switch` in `loop()` (~line 1194), add after `case 4` — this tells the chip to actually run your function when the cycle reaches pattern 5:

```
    case 5:
      redWhiteBluePattern();
      break;
```

Step 4 — name it. In `subModeNames[]` (~line 1179), add `"RED WHITE BLUE"` after `"CUSTOM"` — this gives the pattern a readable name for Serial Monitor.

Step 5 — test. Upload, open Serial Monitor (115200 baud), and wait for `RED WHITE BLUE` to print.

Any of the other samples from Part 2 can be turned into an additional pattern the same way — wrap the code in `void yourPatternName() { ... }` and follow the same 4 steps.

Part 4: Removing Patterns

Don't like one of the built-in patterns? Same 4 spots, in reverse:

1. Delete (or comment out) its `case` in the `switch`
2. **Renumber the remaining cases** so they stay sequential from 0
3. Remove its name from `subModeNames[]`
4. Decrease `NUM_PATTERNS` by 1

To keep only **FLIGHT** running on a loop: set `NUM_PATTERNS 1`, keep just `case 0`, and set `subModeNames[]` to `{ "FLIGHT" }`.

Part 5: Tips and Common Mistakes

- **Use `millis()`, not `delay()`,** for timing (see the examples above) — long `delay()` calls freeze the whole board and make animations choppy.
- **Always call `strip.show()`** after setting pixel colors, or nothing will update.
- **Call `lights()`** (or `lights(false)`) so the nose/wingtip nav lights stay in sync with your pattern.
- **`NUM_PATTERNS` mismatch** is the most common bug when adding a whole new pattern (Part 3) — if it never appears, check it first.
- **Case numbers must be sequential** (0, 1, 2, 3…) with no gaps or duplicates.

Debugging with Serial Monitor

The Serial Monitor isn't just for the initial "is it running" check — it's the easiest way to see what your pattern is actually doing while you're writing it.

- **Picking the right port:** if Tools → Port lists more than one, unplug the board and see which entry disappears — that's the one you want. Arduino IDE 2.x usually labels the port with the board name too.
- **Baud rate must match:** set the Serial Monitor to **115200** (Tools → Serial Monitor, or the icon in the top right) — a mismatched rate just shows garbled text.

- **Add your own debug output:** drop `Serial.print()` / `Serial.println()` calls into any pattern to watch variables change in real time, e.g.:

  ```
  Serial.print("ledIndex: ");
  Serial.println(ledIndex);
  ```

  Gate frequent ones behind a timer so they don't flood the monitor — see how `lastStateDebug` is used in `loop()` for an example.

Study the FLIGHT Pattern

`flightPattern()` (~line 710) is the most advanced pattern in the file — a multi-phase state machine with non-linear (exponential) motion curves and completion-based timing instead of a fixed duration. Worth reading once you're comfortable with the basics above.

Share Your Patterns

Fork the repo, add your patterns, and submit a pull request — help others learn!

See Also

README.md	General info about the board, hardware, and features
SOLDER.md	Photo guide for soldering and gluing the PCB assembly